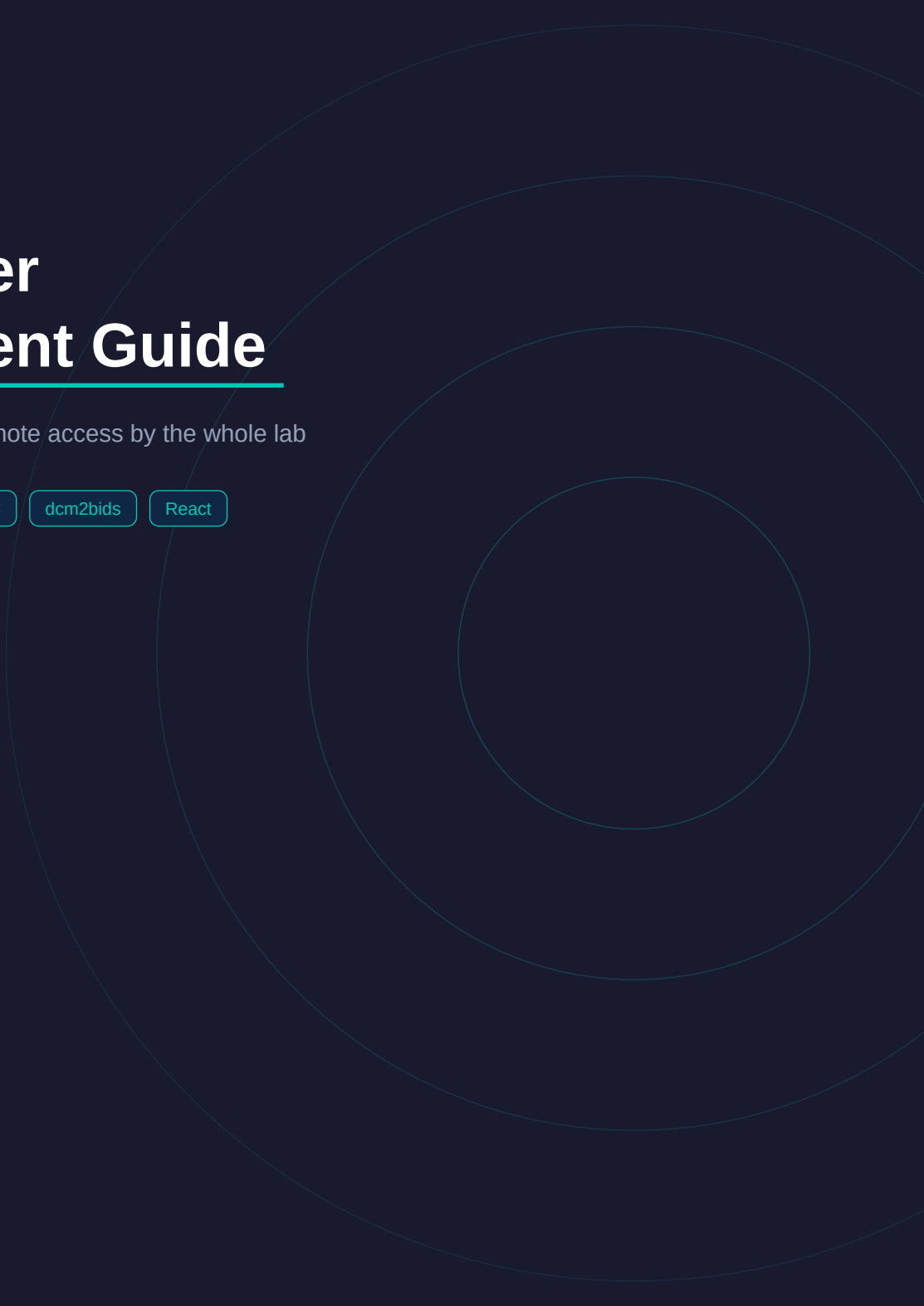




WebMRIQC

Lab Server Deployment Guide

Step-by-step setup for remote access by the whole lab

Docker

FastAPI

MRIQC

dcm2bids

React

Overview

WebMRIQC is a neuroimaging quality-control web application that runs entirely on a single server. It combines a React web interface, a FastAPI backend, and the **MRIQC** pipeline — so any member of the lab can open a browser, upload MRI data, and receive quality metrics and reports without installing any software locally.

This guide covers:

- Installing Docker on the compute server
- Building and starting the WebMRIQC container
- Configuring the port and resources
- Accessing the site from any machine on the network
- Optional HTTPS setup and firewall configuration
- Day-to-day management (start, stop, update, logs)

Server Requirements

Resource	Minimum	Recommended
CPU cores	8 cores	16–32 cores
RAM	16 GB	32–64 GB
Disk (free)	20 GB	60 GB
Operating System	Ubuntu 20.04+ / Debian 11+	Ubuntu 22.04 LTS
Docker	Docker Engine 24+ with Compose v2	Latest stable

Note

MRIQC is memory-intensive. Processing a single T1w subject typically uses 8–16 GB RAM. With 32 GB of RAM and 2 workers, the server can comfortably handle two concurrent jobs.

Step-by-Step Installation

1

Copy the project files to the server

From your local machine, transfer the project folder to the server:

BASH

```
scp -r ./mriqc-web user@YOUR_SERVER_IP:~/webmriqc
# – or – if the project is in a git repository:
git clone <your-repo-url> webmriqc
```

2

SSH into the server and enter the project directory

BASH

```
ssh user@YOUR_SERVER_IP  
cd ~/webmriqc
```

3

Make the deploy script executable

This only needs to be done once:

BASH

```
chmod +x deploy.sh
```

4

Build and start WebMRIQC

A single command handles everything — it installs Docker if missing, builds the container image, and starts the service:

BASH

```
./deploy.sh
```

■ Note

The first run downloads the nipreps/mriqc base image (~5–6 GB). This can take 10–20 minutes depending on internet speed. Subsequent starts are instant.

Once running, open a browser on any machine on the network and go to:

```
http://YOUR_SERVER_IP:8000
```

Replace **YOUR_SERVER_IP** with the server's actual IP address (run `hostname -I` on the server to find it).

Configuration

Create a `.env` file in the project directory to customise the deployment. All settings have sensible defaults — this step is optional.

.ENV

```
# .env – place this file next to docker-compose.yml

# Port the site is reachable on (default: 8000)
PORT=8000

# Number of worker processes (default: 2)
# Only raise above 2 if the server has >= 64 GB RAM
WORKERS=2
```

Variable	Default	Description
PORT	8000	Host port the site is reachable on. Change to 80 to access without a port number.
WORKERS	2	Number of parallel worker processes. Each can run one MRIQC job simultaneously.

After editing .env, apply the changes with:

BASH

```
./deploy.sh --restart
```

Managing the Service

Use the deploy.sh helper script for day-to-day operations:

BASH

```
./deploy.sh           # start (or start after first build)
./deploy.sh --restart # stop, rebuild the image, and restart
./deploy.sh --stop    # stop and remove the container
./deploy.sh --logs    # stream live logs (Ctrl-C to stop)
```

Or use Docker Compose directly:

BASH

```
docker compose ps          # check if container is running
docker compose logs -f webmriqc # live log stream
docker compose down        # stop
docker compose up -d       # start (no rebuild)
docker compose up -d --build # rebuild image and start
```

Updating after code changes

BASH

```
git pull          # pull latest code onto the server
./deploy.sh --restart  # rebuild image and restart
```

Opening the Firewall Port

If the server has a firewall enabled, you need to open the port before other machines can reach the site.

Ubuntu / Debian (UFW)

BASH

```
sudo ufw allow 8000/tcp
sudo ufw reload
sudo ufw status  # confirm port is open
```

CentOS / RHEL / Fedora (firewalld)

BASH

```
sudo firewall-cmd --permanent --add-port=8000/tcp
sudo firewall-cmd --reload
```

Adding HTTPS (Optional)

HTTPS encrypts the connection between users' browsers and the server. It is optional for a local network but recommended if the server is reachable from the internet.

Option A — Self-signed certificate (no domain name needed)

Run these commands on the server inside the project directory:

BASH

```
mkdir -p ssl
openssl req -x509 -nodes -days 3650 -newkey rsa:4096 \
  -keyout ssl/key.pem -out ssl/cert.pem \
  -subj "/CN=webmriqc"
```

Then open `docker-compose.yml` and uncomment the `nginx` service block at the bottom of the file. Restart:

BASH

```
./deploy.sh --restart
```

The site will now be at **https://YOUR_SERVER_IP**. Browsers will show a certificate warning — click *Advanced* → *Proceed* to continue. This is expected for self-signed certificates.

Option B — Automatic certificate via Caddy (requires a domain name)

If the server has a public domain name, install Caddy — it provisions and renews Let's Encrypt certificates automatically:

BASH

```
sudo apt install -y debian-keyring debian-archive-keyring apt-transport-https
curl -1sLf 'https://dl.cloudflare.com/pub/linux/apt/sources.list.d/caddy-stable.list' \
| sudo tee /etc/apt/sources.list.d/caddy-stable.list
sudo apt update && sudo apt install -y caddy
```

Edit /etc/caddy/Caddyfile:

CADDYFILE

```
yourdomain.com {
    reverse_proxy localhost:8000
}
```

BASH

```
sudo systemctl restart caddy
```

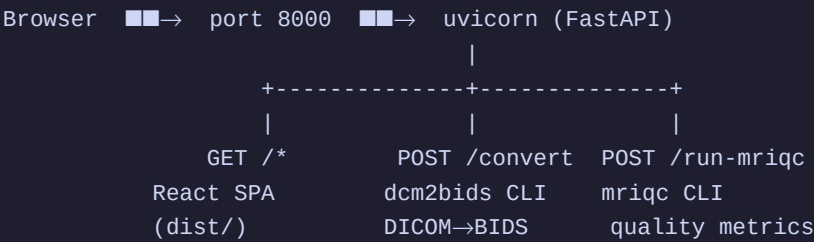
Troubleshooting

Symptom	Likely Cause	Fix
Site unreachable from other computers	Firewall blocking the port	Run: <code>sudo ufw allow 8000/tcp && sudo ufw reload</code>
Container exits immediately after starting	Not enough RAM or disk space	Check logs: <code>docker compose logs webmriqc</code>
MRIQC job hangs for >30 minutes	Out of memory — MRIQC killed by OS	Set <code>WORKERS=1</code> in <code>.env</code> and restart. Consider adding more RAM.
"mriqc not found" on the health check page	First-run Docker image pull still in progress	Wait 2–3 minutes and refresh the page.
Build fails at npm ci step	Network issue downloading npm packages	Retry <code>./deploy.sh --restart</code> once network is stable.
Need to free disk space	Crashed jobs left temp files behind	Run: <code>sudo rm -rf /tmp/webmriqc/*</code>

Architecture Overview

Everything runs inside a single Docker container. There is no external database or separate service to manage.

ARCHITECTURE



Component	Description
React + Vite	The browser UI — served as static files from the dist/ folder inside the container.
FastAPI + uvicorn	The Python web server. Handles file uploads, runs tools, and returns results.
dcm2niix + dcm2bids	Converts raw DICOM data into BIDS format before quality assessment.
MRIQC 24.0.2	The neuroimaging QC pipeline (via nipreps/mriqc Docker base image). Computes 50+ IQMs.